

JAVA 8 – COMPLETE PROFESSIONAL GUIDE

(Structured Book Format – Interview + Practical + Programs)



TABLE OF CONTENTS

1. Introduction to Java 8
2. Lambda Expressions
3. Functional Interfaces
4. Built-in Functional Interfaces
5. Method References
6. Stream API – Fundamentals
7. Stream Intermediate Operations
8. Stream Terminal Operations
9. Collectors Class
10. Parallel Streams
11. Optional Class
12. Default & Static Methods in Interfaces
13. Date & Time API (java.time)
14. Nashorn JavaScript Engine
15. Base64 Encoding
16. Real-World Case Studies

17. Performance & Best Practices

18. Interview Preparation Section

CHAPTER 1: Introduction to Java 8

1.1 Why Java 8 Was Revolutionary

Java 8 introduced:

- Functional Programming support
- Lambda expressions
- Stream API
- New Date/Time API
- Optional class
- Default methods in interfaces

It solved:

- Verbose code
 - Complex iteration logic
 - Null pointer issues
 - Legacy date problems
-

CHAPTER 2: Lambda Expressions

2.1 What is Lambda?

A lambda expression is an anonymous function.

Syntax

(parameters) -> expression

2.2 Basic Example

```
interface MyInterface {  
    void display();  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyInterface obj = () -> System.out.println("Hello Java 8");  
        obj.display();  
    }  
}
```

2.3 Lambda with Parameters

```
interface Add {  
    int sum(int a, int b);  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Add obj = (a, b) -> a + b;  
        System.out.println(obj.sum(10, 20));  
    }  
}
```

2.4 Advantages

- Reduces boilerplate code
- Enables functional programming
- Improves readability

CHAPTER 3: Functional Interfaces

3.1 Definition

An interface with exactly **one abstract method**.

Example

```
@FunctionalInterface
interface Square {
    int calculate(int x);
}
```

3.2 Runnable Example

```
Runnable r = () -> System.out.println("Thread Running");
new Thread(r).start();
```

CHAPTER 4: Built-in Functional Interfaces

4.1 Predicate

```
Predicate<Integer> p = n -> n > 5;
System.out.println(p.test(10));
```

4.2 Function

```
Function<Integer, Integer> f = n -> n * n;
System.out.println(f.apply(4));
```

4.3 Consumer

```
Consumer<String> c = s -> System.out.println(s);  
c.accept("Java");
```

4.4 Supplier

```
Supplier<Double> s = () -> Math.random();  
System.out.println(s.get());
```

CHAPTER 5: Method References

Types

1. Static method reference
 2. Instance method reference
 3. Constructor reference
-

Example

```
List<String> list = Arrays.asList("A", "B", "C");  
list.forEach(System.out::println);
```

CHAPTER 6: Stream API Fundamentals

6.1 What is Stream?

A sequence of elements supporting functional-style operations.

6.2 Creating Streams

```
List<Integer> list = Arrays.asList(1,2,3,4,5);
```

```
Stream<Integer> stream = list.stream();
```

CHAPTER 7: Intermediate Operations

filter()

```
list.stream()  
  .filter(n -> n % 2 == 0)  
  .forEach(System.out::println);
```

map()

```
list.stream()  
  .map(n -> n * 2)  
  .forEach(System.out::println);
```

sorted()

```
list.stream()  
  .sorted()  
  .forEach(System.out::println);
```

distinct()

```
list.stream()  
  .distinct()  
  .forEach(System.out::println);
```

limit() and skip()

```
list.stream()  
  .skip(1)  
  .limit(3)  
  .forEach(System.out::println);
```

CHAPTER 8: Terminal Operations

collect()

```
List<Integer> result = list.stream()  
    .filter(n -> n > 2)  
    .collect(Collectors.toList());
```

count()

```
long count = list.stream().count();
```

reduce()

```
int sum = list.stream()  
    .reduce(0, (a, b) -> a + b);
```

CHAPTER 9: Collectors Class

groupingBy()

```
Map<Integer, List<Integer>> map =  
    list.stream()  
        .collect(Collectors.groupingBy(n -> n % 2));
```

partitioningBy()

```
Map<Boolean, List<Integer>> map =  
    list.stream()  
        .collect(Collectors.partitioningBy(n -> n > 3));
```

CHAPTER 10: Parallel Streams

```
list.parallelStream()  
    .filter(n -> n > 2)  
    .forEach(System.out::println);
```

When to Use?

- Large datasets

- CPU-intensive tasks
 - Stateless operations
-

CHAPTER 11: Optional Class

Avoid NullPointerException

```
Optional<String> name = Optional.ofNullable(null);
System.out.println(name.orElse("Default"));
```

isPresent() Example

```
Optional<String> opt = Optional.of("Java");
if(opt.isPresent()) {
    System.out.println(opt.get());
}
```

CHAPTER 12: Default & Static Methods

```
interface Vehicle {
    default void start() {
        System.out.println("Starting...");
    }
}
```

CHAPTER 13: Date & Time API

```
LocalDate date = LocalDate.now();
LocalTime time = LocalTime.now();
LocalDateTime dt = LocalDateTime.now();
```

```
System.out.println(date);
System.out.println(time);
System.out.println(dt);
```

Formatting Date

```
DateTimeFormatter formatter =  
    DateTimeFormatter.ofPattern("dd-MM-yyyy");  
  
System.out.println(date.format(formatter));
```

CHAPTER 14: Nashorn JavaScript Engine

```
ScriptEngine engine =  
    new ScriptEngineManager().getEngineByName("nashorn");  
  
engine.eval("print('Hello from JS')");
```

CHAPTER 15: Base64 Encoding

```
String encoded = Base64.getEncoder()  
    .encodeToString("Java8".getBytes());  
  
System.out.println(encoded);
```

CHAPTER 16: Real-World Case Study (Employee)

```
class Employee {  
    int id;  
    String name;  
    double salary;  
  
    Employee(int id, String name, double salary) {  
        this.id = id;  
        this.name = name;
```

```
        this.salary = salary;
    }
}
```

Filter High Salary Employees

```
employees.stream()
    .filter(e -> e.salary > 50000)
    .map(e -> e.name)
    .forEach(System.out::println);
```

CHAPTER 17: Performance & Best Practices

- ✓ Prefer streams for data processing
 - ✓ Avoid using peek() in production
 - ✓ Use parallel streams carefully
 - ✓ Avoid modifying shared state
 - ✓ Prefer method references
-

CHAPTER 18: Interview Preparation

Important Questions

1. Difference between map and flatMap
2. What is lazy evaluation?
3. Stateless vs Stateful operations
4. How parallel stream works?
5. What is Optional?
6. Why default methods were introduced?



ADVANCED STREAM PROBLEM

Find highest salary employee:

```
Optional<Employee> max =  
    employees.stream()  
        .max(Comparator.comparing(e -> e.salary));  
  
max.ifPresent(e -> System.out.println(e.name));
```



REDUCE EXAMPLE

```
int factorial = IntStream.rangeClosed(1, 5)  
    .reduce(1, (a, b) -> a * b);  
  
System.out.println(factorial);
```